

Inside the Demo Build

A line-by-line tour of the Makefile, the linker script, and the startup assembly

Renode Lab — Demo (bare-metal ARM Cortex-A9)

June 2026

Contents

Big picture: how three small files become a running program	1
What exactly is the ELF file?	2
Part 1 — The Makefile	4
The toolchain variables	4
The compiler and linker flags	5
The object list	6
The targets and rules	6
Part 2 — The linker script <code>src/link.ld</code>	7
Why we need a linker script at all	7
Line by line	8
Part 3 — The startup assembly <code>src/start.S</code>	10
Why we need startup code	10
Line by line	11
How it all ties together at run time	12
Abbreviations	13

Big picture: how three small files become a running program

A bare-metal program has *no operating system underneath it*. There is no loader to set up a stack, no C runtime to zero your global variables, and no shell to hand control to `main()`. Everything that a normal program takes for granted, **you** must arrange. In this Demo that job is split across three files, each answering one question:

File	Question it answers	Tool that consumes it
Makefile	<i>How</i> do I turn source into a binary?	make (drives the compiler)

File	Question it answers	Tool that consumes it
src/link.ld	<i>Where</i> in memory does each piece go?	the linker (ld, via gcc)
src/start.S	<i>What</i> runs first, before main()?	the assembler, then the CPU

The build runs in three stages. Read each stage top-to-bottom; the output of one stage is the input to the next.

Stage 1 – compile / assemble each source into an object file (.o):

```
src/start.S  →  src/start.o
src/main.c   →  src/main.o
```

Stage 2 – link the objects into one ELF, using link.ld to assign addresses:

```
src/start.o + src/main.o  —(link.ld)→  sw/bare.elf
```

Stage 3 – (optional) strip the ELF down to a raw memory image:

```
sw/bare.elf  —(objcopy)→  sw/bare.bin
```

What happens in each stage:

- **Stage 1 — compile/assemble.** The compiler turns `main.c`, and the assembler turns `start.S`, into *object files* (.o): machine code where addresses are still placeholders (“relocations”) to be filled in later.
- **Stage 2 — link.** The linker glues the object files into a single program and, guided by `link.ld`, assigns every byte its **final address**, producing an **ELF** executable (`sw/bare.elf`).
- **Stage 3 — objcopy.** Optionally strips the ELF down to a raw **binary** image (`sw/bare.bin`) — just the bytes, no metadata — the form you’d write to real flash.

Renode loads the **ELF** (not the .bin), because the ELF still carries the entry-point address, the load addresses, and the symbol names. The CPU then begins executing at the entry point — the first instruction of `start.S`.

What exactly is the ELF file?

`bare.elf` is the central artifact of the build, so it is worth understanding what it actually is.

ELF stands for **Executable and Linkable Format**. It is the standard container format for compiled programs on Linux, most Unix systems, and nearly every embedded cross-toolchain (including `arm-none-eabi-`). It is the linker’s normal output.

What it is *for*. An ELF file is not just raw machine code — it is the code *plus a map* describing what the code is and where every piece belongs in memory. That map is what lets other tools do their job without guessing:

- a **loader** (or, on bare metal, a flashing tool) knows which bytes to place at which physical addresses;
- a **debugger** (GDB) can map an address back to a function name and a source line;
- a **simulator** like Renode can read the entry point, lay the sections into the emulated memory, and print symbol names while the program runs (this is exactly what `LogFunctionNames` relies on).

What it contains. An ELF file is organized into a header plus several tables and sections:

Part	What it holds	Who uses it
ELF header	Magic bytes 7F 45 4C 46 (<code>\x7F ELF</code>), 32- vs 64-bit class, endianness, target machine (ARM), and the entry-point address	loaders, Renode
Program headers (segments)	How to load the file: which chunks go to which memory addresses, with sizes and R/W/X flags	loaders/flashers
Section headers (sections)	The named pieces: <code>.text</code> (code), <code>.rodata</code> (constants), <code>.data</code> (initialized globals), <code>.bss</code> (zero-init globals)	linker, debugger
Symbol table (<code>.symtab</code>)	Names ↔ addresses: <code>_start</code> , <code>main</code> , <code>_stack_top</code> , <code>_bss_start</code> , ...	linker, GDB, Renode
Debug info (<code>.debug_*</code> , DWARF)	Address → source-file/line mapping, variable types — emitted because we pass -g	GDB, Renode

ELF vs the raw `.bin`. The `.bin` produced in Stage 3 is *only* the bytes that sit in memory — it has **no** addresses, symbols, or debug info. It is smaller and is what you'd burn to physical flash, but a tool loading it must be *told* where it belongs. The ELF carries all of that context with it, which is why this lab hands Renode the ELF: `sysbus LoadELF @sw/`

`bare.elf` places each section at the address `link.ld` chose and registers the symbols, so the emulator knows to start at `_start` and can name functions as they execute.

The rest of this document walks every line of all three files.

Part 1 — The Makefile

The Makefile is a recipe book. Each *rule* says “to build target X, which inputs are needed, and which shell commands produce it.” `make` reads the timestamps and only rebuilds what changed.

Here is the complete file, followed by a line-by-line explanation.

```
CROSS    ?= arm-none-eabi-
CC       := $(CROSS)gcc
AS       := $(CROSS)as
LD       := $(CROSS)ld
OBJCOPY  := $(CROSS)objcopy

CFLAGS   := -mcpu=cortex-a9 -marm \
            -ffreestanding -nostdlib -nostartfiles \
            -Wall -O2 -g
LDFLAGS  := -T src/link.ld -nostdlib

OBJS     := src/start.o src/main.o

all: sw/bare.elf sw/bare.bin

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

%.o: %.S
    $(CC) $(CFLAGS) -c $< -o $@

sw/bare.elf: $(OBJS)
    mkdir -p sw
    $(CC) $(CFLAGS) $(LDFLAGS) -o $@ $(OBJS)

sw/bare.bin: sw/bare.elf
    $(OBJCOPY) -O binary $< $@

clean:
    rm -f src/*.o sw/bare.elf sw/bare.bin

.PHONY: all clean
```

The toolchain variables

```
CROSS    ?= arm-none-eabi-
```

Defines the **cross-compiler prefix**. We are compiling *on* an x86/ARM host but *for* a bare-metal ARM target, so we cannot use the system `gcc`; we use the `arm-none-eabi- toolchain` (ARM architecture, `none` = no OS, `eabi` = the embedded ABI). The `?=` operator means “assign only if not already set,” so you can override it from the command line, e.g. `make CROSS=arm-linux-gnueabihf-`, without editing the file.

```
CC      := $(CROSS)gcc
```

The **C compiler** (and, as we use it here, the linker driver). Expands to `arm-none-eabi-gcc`. `:=` is *immediate assignment*: the right-hand side is expanded once, now, rather than re-expanded on every use.

```
AS      := $(CROSS)as
```

The **assembler**, `arm-none-eabi-as`. Declared for completeness/clarity. In this project we actually assemble `.S` files *through* `gcc` (so the C preprocessor runs first), but naming the tool documents the toolchain.

```
LD      := $(CROSS)ld
```

The **linker**, `arm-none-eabi-ld`. Again declared for clarity; we invoke the linker *via* `gcc` so the compiler can add the bits of glue it knows about.

```
OBJCOPY := $(CROSS)objcopy
```

objcopy converts between object/binary formats — here, `ELF` → raw binary.

The compiler and linker flags

```
CFLAGS := -mcpu=cortex-a9 -marm \
         -ffreestanding -nostdlib -nostartfiles \
         -Wall -O2 -g
```

`CFLAGS` are the options passed to the compiler. The trailing `\` is a line continuation — this is one logical line. Each flag:

- `-mcpu=cortex-a9` — generate code tuned for the **Cortex-A9** core, exactly the CPU the Renode platform models. This selects the instruction set and scheduling for that part.
- `-marm` — emit the **32-bit ARM** instruction set (as opposed to `-mthumb`, the compressed Thumb encoding). Keeps the startup story simple.
- `-ffreestanding` — tell the compiler this is a **freestanding** environment: no hosted C library, `main` is not special, and it must not assume standard library semantics. (The opposite is `-fhosted`.)
- `-nostdlib` — **do not link the standard libraries** (`libc`, `libgcc` startup, etc.). We supply our own startup and use no `libc`.
- `-nostartfiles` — **do not pull in the default C runtime startup objects** (`crt0.o/crti.o...`). Those would normally call `main` for you; here *our* `start.S` does that job.
- `-Wall` — enable the common **warnings**. Cheap insurance against bugs.
- `-O2` — **optimization level 2**: a good speed/size balance.

- `-g` — emit **debug information** (DWARF). This is what lets Renode and GDB map addresses back to source lines and symbol names.

```
LDFLAGS := -T src/link.ld -nostdlib
```

LD_FLAGS are options for the **link** step:

- `-T src/link.ld` — use **our linker script** instead of the toolchain’s built-in default. This is the crucial flag that places our code at `0x80000000` to match the hardware. (`-T` = “use this linker *Template/* script.”) Without it the linker would pick default addresses that the Renode platform does not have RAM at, and nothing would run.
- `-nostdlib` — repeated at link time so the linker also skips the standard libraries and their default startup.

The object list

```
OBJS := src/start.o src/main.o
```

The list of **object files** that make up the program. Order matters a little: `start.o` is listed first, and combined with the linker script it ensures the reset code lands at the very start of RAM (the CPU’s entry point).

The targets and rules

```
all: sw/bare.elf sw/bare.bin
```

The **default target** (the first one in the file). Running `make` with no arguments builds `all`, whose *prerequisites* are the ELF and the raw binary. `all` has no recipe of its own; it just depends on the real outputs.

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

A **pattern rule**: “any `foo.o` can be built from the matching `foo.c`.” The `%` is the wildcard stem. The recipe line **must begin with a real TAB**, not spaces — that is how `make` distinguishes a recipe from other text.

- `$(CC) $(CFLAGS)` — the compiler and our flags.
- `-c` — **compile/assemble only, do not link**; produce an `.o`.
- `$<` — automatic variable for the **first prerequisite** (the `.c` file).
- `-o $@` — output to `$@`, the automatic variable for the **target** (the `.o` file).

```
%.o: %.S
    $(CC) $(CFLAGS) -c $< -o $@
```

The same idea for **assembly** sources. Using a capital `.S` (not `.s`) means the C **preprocessor runs first**, so the assembly file may use `#include`, `#define`, and `/* comments */`. Driving it through `$(CC)` rather than `$(AS)` makes that preprocessing automatic.

```
sw/bare.elf: $(OBJJS)
    mkdir -p sw
    $(CC) $(CFLAGS) $(LDFLAGS) -o $@ $(OBJJS)
```

The **link** rule. `sw/bare.elf` depends on both object files; if either changes, it relinks.

- `mkdir -p sw` — make sure the output directory exists (`-p` = no error if it already does).
- `$(CC) $(CFLAGS) $(LDFLAGS) -o $@ $(OBJJS)` — invoke `gcc` as the **linker driver**: it applies the linker script from `LDFLAGS`, links the objects, and writes `sw/bare.elf`. Letting `gcc` drive the link (instead of calling `ld` directly) lets it add the small compiler-support pieces it knows about.

```
sw/bare.bin: sw/bare.elf
    $(OBJCOPY) -O binary $< $@
```

Produce a **raw binary** from the ELF.

- `$(OBJCOPY)` — the format-conversion tool.
- `-O binary` — **output format = raw binary**: strip away ELF headers, symbols, and section metadata, leaving just the bytes that would sit in memory. This is the form you’d flash to real hardware. (Renode prefers the ELF, which still carries symbols and load addresses.)
- `$<` (the ELF) → `$@` (the `.bin`).

```
clean:
    rm -f src/*.o sw/bare.elf sw/bare.bin
```

Housekeeping target. `make clean` deletes the build artifacts so the next build starts fresh. `-f` keeps `rm` quiet if the files are already gone.

```
.PHONY: all clean
```

Declares `all` and `clean` as **phony** — names that are not real files. Without this, if a file named `clean` ever appeared in the directory, `make` would think the target was “up to date” and skip the recipe. `.PHONY` forces the recipe to run every time.

Part 2 — The linker script `src/link.ld`

Why we need a linker script at all

When the linker combines your object files it must decide the **final memory address of every function and variable**. On a hosted system the OS loader and a default linker script handle this; the program is loaded wherever the OS says. On bare metal **there is no loader** — the program must already be linked for the *exact physical addresses where the hardware has memory*. If the code expects to live at `0x80000000` but is linked for `0x0`, every absolute address (function pointers, the stack top, jump targets) is wrong and the program crashes immediately.

The linker script answers three questions:

1. **Where is the memory?** (the MEMORY block)
2. **Where does each section go?** (the SECTIONS block)
3. **What symbol addresses does the startup code need?** (`_bss_start`, `_bss_end`, `_stack_top`)

Those addresses must match the Renode platform, which maps system RAM at `0x80000000` with a size of `0x04000000` (64 MiB).

Here is the complete script:

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 64M
}

SECTIONS {
    . = ORIGIN(RAM);

    .text : {
        KEEP(*(.text.start))
        *(.text*)
    } > RAM

    .rodata : { *(.rodata*) } > RAM
    .data   : { *(.data*) } > RAM

    . = ALIGN(4);
    _bss_start = .;
    .bss : { *(.bss*) *(COMMON) } > RAM
    . = ALIGN(4);
    _bss_end = .;

    . = ALIGN(16);
    . = . + 0x4000;
    _stack_top = .;
}
```

Line by line

`ENTRY(_start)`

Declares the program's **entry point** — the symbol whose address goes into the ELF header as “start executing here.” Renode reads this and sets the CPU's program counter to `_start` (defined in `start.S`) after loading.

```
MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 64M
}
```

The **MEMORY** block describes the physical memory regions available.

- RAM — a name we chose for this region.
- (rwx) — its **attributes**: readable, writable, executable.
- ORIGIN = 0x80000000 — the region's **start address**, matching the systemram mapping in smarttimer_arm.repl.
- LENGTH = 64M — its **size**, 64 MiB, matching the platform's 0x04000000.

If these numbers did not match the .repl, the program would be linked for addresses that don't exist in the emulated machine.

SECTIONS {

Opens the **SECTIONS** block, which lays out the output sections and assigns their addresses, in order.

```
. = ORIGIN(RAM);
```

The . symbol is the **location counter** — “the current address.” We set it to the start of RAM (0x80000000), so the first section begins exactly there.

```
.text : {
    KEEP(*(.text.start))
    *(.text*)
} > RAM
```

The .text output section holds executable **code**.

- KEEP(*(.text.start)) — place the .text.start input section **first**, and KEEP it even if link-time garbage collection thinks it is unused. This is the special section start.S puts _start into, guaranteeing the reset vector sits at the very base of RAM (the entry address).
- *(.text*) — then gather the .text (and .text.*) sections from **all** input object files (the * before (matches every file). That is the rest of the code, including main.
- > RAM — **place this section in the RAM region**.

```
.rodata : { *(.rodata*) } > RAM
```

.rodata = read-only data: string literals and const globals. Collected from all objects and placed next in RAM.

```
.data : { *(.data*) } > RAM
```

.data = initialized read/write globals (e.g. int x = 5;). Their initial values live here in the image.

```
. = ALIGN(4);
_bss_start = .;
```

- . = ALIGN(4) — bump the location counter up to the next **4-byte boundary** so the next section is word-aligned (the startup code clears .bss a 32-bit word at a time).
- _bss_start = .; — **define a symbol** equal to the current address. The startup code reads this to know where .bss begins.

```
.bss : { *(.bss*) *(COMMON) } > RAM
```

.bss = **uninitialized** globals (e.g. `int counter;`). They are defined to start as zero. To save space they occupy *no bytes in the image*; only their address range is reserved, and the startup code zeroes it at runtime.

- `*(.bss*)` — all `.bss` sections.
- `*(COMMON)` — the “common” symbols (tentative definitions) the compiler may emit; conventionally swept into `.bss`.

```
. = ALIGN(4);
_bss_end = .;
```

Align again, then define `_bss_end` at the address just past `.bss`. Now `start.S` has the pair `[_bss_start, _bss_end)` it needs to zero exactly the right range.

```
. = ALIGN(16);
. = . + 0x4000;
_stack_top = .;
```

Carve out the **stack**.

- `. = ALIGN(16)` — align to 16 bytes (the ARM AAPCS requires the stack to be 8-byte aligned at public function boundaries; 16 is a safe, common choice).
- `. = . + 0x4000` — **advance the location counter by 16 KiB**, reserving that much space for the stack.
- `_stack_top = .;` — define `_stack_top` at the *high* end of that block. ARM stacks grow **downward**, so the stack pointer is initialised to the top and descends into the reserved region. `start.S` loads this into `sp`.

```
}
```

Closes the `SECTIONS` block. The image layout, from `0x80000000` upward, is now: code → read-only data → initialized data → (reserved) `bss` → (reserved) `stack`, with the three symbols the startup code depends on defined along the way.

Part 3 — The startup assembly `src/start.S`

Why we need startup code

When the Cortex-A9 comes out of reset it is in a very raw state: the program counter points at the reset address (here, the base of RAM where `_start` lives), but **the stack pointer is not valid** and **.bss is full of garbage**. C code cannot run yet — the very first function call needs a stack, and any zero-initialized global would hold junk. The compiler does *not* emit this setup for a freestanding target (we even passed `-nostartfiles` to suppress the default version). So we write the minimal “`crt0`” ourselves:

1. set up the stack pointer,
2. zero the `.bss` section,

3. call `main()`,
4. and park the core safely if `main()` ever returns.

Here is the complete file:

```

.section .text.start, "ax"
.global _start
_start:
    ldr sp, =_stack_top

    ldr r0, =_bss_start
    ldr r1, =_bss_end
    mov r2, #0
1:  cmp r0, r1
    bcs 2f
    str r2, [r0], #4
    b 1b

2:  bl  main
hang:
    wfi
    b  hang

```

Line by line

```

.section .text.start, "ax"

```

An **assembler directive** placing the following code into a section named `.text.start`. The "ax" flags mark it **allocatable** and **executable**. The linker script deliberately puts this section *first* in `.text`, so `_start` ends up at the base of RAM — the address the CPU jumps to on reset.

```

.global _start

```

Make the `_start` symbol **global** (visible to the linker) so that `ENTRY(_start)` in the linker script can resolve to it.

```

_start:

```

The **label** marking the entry point — the first instruction executed.

```

    ldr sp, =_stack_top

```

Set up the stack. Load the address of `_stack_top` (from the linker script) into the **stack pointer** `sp`. The `=symbol` syntax is an *assembler literal pool* load: it places the 32-bit address in a nearby constant pool and loads it. After this line, C function calls (which push/pop the stack) are safe.

```

    ldr r0, =_bss_start
    ldr r1, =_bss_end
    mov r2, #0

```

Prepare to **zero** `.bss`:

- `r0` $\leftarrow$ address of `_bss_start` (the cursor we'll walk).
- `r1` $\leftarrow$ address of `_bss_end` (the stop point).
- `r2` $\leftarrow$ the constant 0, the value we'll store.

```
1:  cmp  r0, r1
```

A **local numeric label** 1: (the loop top). `cmp r0, r1` computes `r0 - r1` and sets the condition flags **without** storing the result — it's just a comparison of the current cursor against the end.

```
    bcs  2f
```

Branch if Carry Set to the forward label 2. For an unsigned `cmp`, carry-set means `r0 >= r1`, i.e. we've reached (or passed) the end of `.bss`, so we exit the loop. `2f` = "the 2: label *forward* from here."

```
    str  r2, [r0], #4
```

Store the zero in `r2` to the address in `r0`, then **post-increment** `r0` by 4 (`[r0], #4` writes to `[r0]` and *afterwards* adds 4 to `r0`). So this clears one 32-bit word and advances the cursor.

```
    b    1b
```

Branch back to label 1 (`1b` = "the 1: label *backward*"), repeating the loop until the `bcs` exit fires. The result: every word in `[_bss_start, _bss_end)` is set to zero.

```
2:  bl   main
```

The loop's exit label. `bl main` = **branch with link**: it jumps to `main` and saves the return address in the link register (`lr`), so a normal `main` could return here. This is the hand-off into C.

```
hang:
    wfi
    b    hang
```

The **safe parking loop**, in case `main()` ever returns (ours returns 0):

- `hang`: — label for the loop.
- `wfi` — **Wait For Interrupt**: puts the core into a low-power idle until an interrupt arrives. On bare metal with nothing else to do, this avoids spinning the CPU at full tilt.
- `b hang` — loop back, so even if `wfi` wakes spuriously we stay parked forever rather than executing whatever bytes follow in memory.

How it all ties together at run time

1. Renode loads `bare.elf`, reads `ENTRY(_start)`, and sets the program counter to the base of RAM where `start.S` placed `_start`.
2. `_start` sets `sp` to `_stack_top`, zeroes `[_bss_start, _bss_end)`, and calls `main()`.

3. `main()` (in `main.c`) reads and writes the SmartTimer MMIO registers, then returns 0.
4. Control falls into the `wfi/b hang` loop, and the core idles.

Every address used in steps 1–4 — the entry point, the stack top, the bss bounds, the MMIO base — is fixed by the linker script to match the memory map declared in `smartrtimer_arm.repl`. That agreement between the **Makefile** (how it's built), the **linker script** (where it lives), and the **startup code** (what runs first) is exactly what makes a bare-metal program work.

Abbreviations

Acronyms and shorthand used throughout this document.

Abbreviation	Stands for	In one line
ABI	Application Binary Interface	The binary-level contract (calling convention, layout) between compiled pieces.
EABI	Embedded ABI	The bare-metal flavour of the ABI; the <code>eabi</code> in <code>arm-none-eabi-</code> .
AAPCS	ARM Architecture Procedure Call Standard	ARM's rules for passing arguments and aligning the stack.
ARM	Advanced RISC Machines	The CPU architecture this Demo targets (Cortex-A9).
RISC	Reduced Instruction Set Computer	CPU design philosophy behind ARM.
BSS	Block Started by Symbol	The section for zero-initialized globals; reserved, not stored in the image.
CPU	Central Processing Unit	The processor core executing the program.
DWARF	(debug format paired with ELF)	The debug-info format emitted by <code>-g</code> ; maps addresses to source.
ELF	Executable and Linkable Format	The linker's output: code plus a memory/symbol map.

Abbreviation	Stands for	In one line
GCC	GNU Compiler Collection	The compiler suite; arm-none-eabi-gcc here.
GDB	GNU Debugger	The debugger that consumes ELF symbols and DWARF.
IRQ	Interrupt Request	A hardware signal asking the CPU to handle an event.
I/O	Input / Output	Communication between the CPU and devices.
ISA	Instruction Set Architecture	The set of instructions a CPU understands (e.g. ARM vs Thumb).
KiB / MiB	Kibibyte / Mebibyte	1 KiB = 1024 bytes; 1 MiB = 1024 KiB.
LR	Link Register	ARM register holding a function's return address (set by bl).
MMIO	Memory-Mapped I/O	Talking to peripherals via loads/stores to fixed addresses.
PC	Program Counter	Register holding the address of the next instruction.
RAM	Random-Access Memory	Read/write working memory (mapped at 0x80000000 here).
SP	Stack Pointer	Register pointing at the top of the stack.
SVC	Supervisor (mode/Call)	The privileged ARM mode the core boots into.

Abbreviation	Stands for	In one line
WFI	Wait For Interrupt	Instruction that idles the core until an interrupt arrives.
crt0	C Run-Time (object zero)	The startup stub that prepares the C environment; start.S is our hand-written crt0.